

Revisão de Conceitos Fundamentais de Orientação a Objetos

Estudo dirigido — Loja de Consoles (Interface, Composição e Herança)

Disciplina	GCVA — Gestão do Ciclo de Vida da Aplicação
Linguagem	Java
Domínio	Loja de consoles (Nintendo, Playstation, Playstation Portátil, Xbox)
Arquivo de partida	Atividade_POO_Problema.java

Objetivo: evoluir, em 5 etapas guiadas, o arquivo `Atividade_POO_Problema.java` até uma versão que aplica corretamente encapsulamento, construtores, composição, interfaces, herança e polimorfismo.

Contexto

O arquivo `Atividade_POO_Problema.java`, fornecido junto com este roteiro, contém uma versão propositalmente simplificada e "malfeita" do sistema de vendas de consoles de uma loja. Ele compila e funciona, mas ignora praticamente todos os pilares da Orientação a Objetos: os atributos são públicos, não há construtores, não há herança nem polimorfismo — apenas um campo de texto (tipo) verificado por uma sequência de `if/else`.

Sua tarefa é evoluir esse arquivo em 5 etapas, cada uma revisando um conceito fundamental de OO. Ao final, o código terá uma estrutura muito mais próxima do que se espera de um sistema orientado a objetos bem projetado — e que servirá de base para os princípios SOLID que vocês já viram em outro material desta disciplina.

🕒 ORIENTAÇÃO AO ESTUDANTE

Abra o arquivo `Atividade_POO_Problema.java` no seu editor/IDE. Ele já compila e roda. Siga os **TODOs** numerados que estão dentro do próprio arquivo, na ordem das 5 partes abaixo — cada parte corresponde a um grupo de **TODOs**. Não pule etapas: cada uma depende do resultado da anterior.

PARTE 1 /5 Encapsulamento

O conceito

Encapsulamento significa proteger o estado interno de um objeto, controlando como ele pode ser lido e alterado. No arquivo de partida, os atributos de **Console** são públicos:

```
public String nome;  
public String tipo;  
public double preco;
```

Isso permite que qualquer parte do programa altere esses valores livremente — inclusive para valores inválidos (por exemplo, um **preço** negativo) — sem que a classe tenha qualquer chance de impedir isso.

Sua tarefa (TODOs 1.1 e 1.2)

- **nome**, **tipo** e **preço** ficam privados.
- Getters (**getNome()**, **getTipo()**, **getPreço()**) são criados para leitura externa.

Pergunta de checagem

Depois de tornar os atributos privados, o código do **Main** (que hoje faz **nintendo.nome = "..."**) ainda compila sem nenhuma outra mudança? Por quê?

PARTE 2 /5 Construtores

O conceito

Um construtor é responsável por garantir que um objeto nasça em um estado válido e completo. Sem um construtor adequado, é possível criar um **Console** "pela metade" — por exemplo, esquecendo de definir o **preço**, que ficaria silenciosamente em **0.0**, um bug difícil de perceber.

Sua tarefa (TODOs 2.1 e 2.2)

- **Console** passa a exigir **nome**, **tipo** e **preço** como parâmetros obrigatórios.
- **main()** passa a criar os três consoles usando esse novo construtor.

Pergunta de checagem

Depois dessa mudança, ainda é possível criar um **Console** sem informar o **preço**? O que aconteceria se você tentasse escrever **new Console("Nintendo Switch", "nintendo")** (faltando um parâmetro)?

PARTE 3 /5 Interface e Composição

O conceito

A classe **Console** hoje mistura três "tipos" de console usando o atributo **tipo** (uma **String**) — um sinal de que deveríamos ter classes específicas para cada tipo. Em vez de criar uma superclasse abstrata para elas herdarem, vamos usar duas ferramentas diferentes: uma **interface** (para definir o contrato comum) e composição (para reaproveitar dados comuns sem herança).

O contrato que toda classe de console vai precisar cumprir:

```
interface IConsole {  
    void ligar();  
    double calcularPreço();  
    String getNome();  
}
```

Sua tarefa (TODOs 3.1 a 3.4)

- **IConsole** é criada com os três métodos acima.
- **DadosConsole** é criada com **nome** e **preçoBase** privados, construtor e getters — ela representa o que todo console tem em comum.

- **Nintendo** e **Playstation** implementam **IConsole** e cada uma tem um atributo do tipo **DadosConsole** (composição), delegando a ele **getNome()** e **getPrecoBase()**.
- A mensagem de **ligar()** e o percentual de **calcularPreco()** de cada uma (10% Nintendo, 20% Playstation) saem do if/else da Loja e vão para dentro de cada classe.

Pergunta de reflexão

Por que usar uma classe separada (**DadosConsole**) em vez de colocar **nome/precoBase** diretamente dentro de **Nintendo** e **Playstation**?

PARTE 4 /5 Herança (usada de forma apropriada)

O conceito

Herança faz sentido quando uma subclasse realmente especializa uma superclasse, sem quebrar nenhum comportamento herdado. É diferente do que vocês viram no material de SOLID (**Solucao_P1**), onde **PlaystationPortatil** sobrescrevia **jogarDisco()** apenas para recusar o comportamento:

```
// Exemplo do material de SOLID (evitar):
public void jogarDisco() {
    throw new UnsupportedOperationException();
}
```

Isso viola o Princípio de Substituição de Liskov (LSP): a subclasse deixa de cumprir uma promessa da superclasse. Nesta etapa, vocês vão criar uma herança que NÃO tem esse problema.

Sua tarefa (TODOs 4.1 a 4.3)

- **PlaystationPortatil** passa a ser **extends Playstation** (em vez de implementar **IConsole** do zero). Isso exige tornar o atributo de **DadosConsole** em **Playstation** protected.
- **ligar()** é sobrescrito (**@Override**) com a mensagem específica da versão portátil.
- **calcularPreco()** também é sobrescrito, usando um percentual próprio (por exemplo, 15% em vez dos 20% herdados).

Pergunta de reflexão

PlaystationPortatil está sobrescrevendo DOIS métodos herdados de **Playstation**. Isso é um problema? Qual a diferença entre sobrescrever um método para mudar SEU CÁLCULO e sobrescrever um método para RECUSAR fazer o que ele promete (como no exemplo de **jogarDisco()** acima)?

PARTE 5 /5 Polimorfismo e Extensibilidade (OCP)

O conceito

Polimorfismo é a capacidade de tratar objetos de tipos diferentes de maneira uniforme, através de uma interface comum, deixando que cada objeto "saiba" executar sua própria versão do comportamento. É isso que vai permitir estender o sistema com um novo tipo de console sem alterar a classe Loja.

Sua tarefa (TODOs 5.1 a 5.5)

1. **venderConsole()** da **Loja** passa a receber um **IConsole** e apenas chamar **console.ligar()** e **console.calcularPreco()**, sem nenhum **if/instanceof**.

2. Crie `venderVarios(List<IConsole> consoles)` em `Loja`, chamando `venderConsole()` para cada item da lista.
3. Crie `calcularFaturamentoTotal(List<IConsole> consoles)` em `Loja`, somando `calcularPreco()` de todos os itens.
4. Crie uma nova classe `Xbox` implementando `IConsole` (com seu próprio percentual, por exemplo 18%) — sem alterar nenhuma linha de `Loja`.
5. No `Main`, monte uma lista com os consoles existentes, chame os dois métodos novos, depois adicione o `Xbox` à lista e chame os dois métodos de novo.



Desafio final

- O que precisou mudar em `Loja` para o `Xbox` passar a funcionar?
- O que isso demonstra sobre o Princípio Aberto/Fechado (OCP)?

Para ir além



CONEXÃO COM O SOLID

O que vocês fizeram aqui — introduzir interface, composição, herança segura e polimorfismo — é exatamente a base sobre a qual os princípios SOLID são construídos. Comparem sua solução final com o material "Solucao_P1" desta disciplina: lá, a mesma ideia de loja de consoles aparece com uma injeção de dependência (DIP) e uma separação de responsabilidades (SRP) que vocês agora têm todo o repertório de OO para entender de verdade.

Boa atividade!